The Hippie Scientist

Full Deep Audit Report — April 4, 2026

Executive Summary

Severity	#	Area	Status
■ Critical	6	Data corruption, broken routes, missing blog content	Fix immediately
■ Significant	6	Duplicate herbs, degenerate slugs, SEO gaps	Fix this week
■ Architecture	4	Deploy pipeline, ops bloat, slug mismatch	Fix this sprint

■ Critical Bugs

1. nan.json herb still present in herbs-summary.json

Two entries in **public/data/herbs-summary.json** have **name: 'nan'** — a recurring pandas CSV serialization artifact. One is a literal **nan.json** file for Acacia confusa (should be **acacia-confusa.json**). The other is Achillea millefolium with its **name** and **common** fields both corrupted to the string "nan". These generate broken page routes (**/herbs/nan**) and corrupt search results, herb browsers, and SEO metadata.

Files affected:

```
public/data/herbs-detail/nan.json → rename to acacia-confusa.json
```

```
public/data/herbs-summary.json → patch name/common fields for achillea-millefolium entry
```

2. [object Object] compound file

public/data/compounds-detail/object-object.json has **name: "[object Object]"** — a JavaScript serialization artifact. This will appear in compound lists, search autocomplete, and the sitemap. Should be deleted immediately.

```
Fix: delete public/data/compounds-detail/object-object.json
```

3. Degenerate single-character / truncated compound slugs

Six compound files have slugs that are either single characters or obviously truncated names — artifacts from the slug normalization stripping too aggressively (e.g. Greek letter prefixes stripped, leaving just "5" or "n"). These create garbage compound pages that damage site credibility.

Delete these files:

```
public/data/compounds-detail/5.json
public/data/compounds-detail/n.json
public/data/compounds-detail/alpha.json
public/data/compounds-detail/beta.json
public/data/compounds-detail/meo.json
public/data/compounds-detail/p.json
```

4. Three orphaned page components — never routed

Three full page components exist in **src/pages/** but have no matching route in **App.tsx**. They are completely unreachable — invisible to users and search engines:

src/pages/HerbBlender.tsx — A complete herb blending UI

src/pages/CompoundIndex.tsx — A dedicated compound index/browse page

src/pages/Lesson.tsx — A lesson detail page for the learning paths feature

Fix: either add routes in App.tsx or delete the components if abandoned.

5. Blog pipeline only publishing 15 of 75 posts

You have **75 blog content files** in content/blog/ (.mdx and .md), but only **15 posts** appear in **public/blogdata/posts/** and only **16 /blog/ URLs** are in the sitemap. That is an 80% loss of blog content. The **build:blog** script is not processing the full content/blog/ directory. This is likely your single highest-ROI fix — potentially 60 new indexed pages from work already done.

```
npm run build:blog → should process all *.mdx and *.md in content/blog/
```

6. publication-manifest.json structure mismatch

The publication manifest wraps entities under **manifest.entities.herbs[]** and **manifest.entities.compounds[]**, but scripts reading it as a flat array will see **0 published entities**. This causes the publish verification step to silently pass with a false zero count. Audit **scripts/verify-publishing.mjs** and confirm it reads **manifest.entities.herbs** not the root array.

■ Significant Problems

7. 465 herbs published out of 677 total herb-detail files (31% gap)

The quality gate correctly passes 465 herbs into the publication index, but ~212 herb-detail files are filtered out. Many of these are likely real herbs with solid data that fail on a single threshold — description under 20 chars, or 0 sources. A targeted audit of the unpublished 212 would likely surface 50–100 near-ready entries that just need a small data patch rather than full enrichment.

8. LSD, THC, CBD, psilocybin, mescaline stored as 'herbs'

Five pure chemical compounds are stored in **public/data/herbs-detail/** and appear in the herb browser and sitemap. These are not botanical herbs. Taxonomically confusing for SEO and credibility — a journalist or researcher landing on your herbs page seeing LSD next to Passionflower will lose confidence. They should be in compounds-detail or explicitly flagged as pure compounds.

```
public/data/herbs-detail/lsd.json → move to compounds-detail/
public/data/herbs-detail/thc.json → move to compounds-detail/
public/data/herbs-detail/cbd.json → move to compounds-detail/
public/data/herbs-detail/psilocybin.json → move to compounds-detail/
public/data/herbs-detail/mescaline.json → move to compounds-detail/
```

9. Duplicate herb entries via common-name alias files

Multiple herbs have two separate JSON files — one for the Latin name and one for the common name — creating duplicate pages with split link equity. Examples identified:

```
ashwagandha.json / withania-somnifera.json
blue-lotus.json / blue-lotus-nymphaea-caerulea.json
bacopa.json / bacopa-monnieri.json
caapi.json / banisteriopsis-caapi.json
damiana.json / turnera-diffusa.json
mugwort.json / artemisia-vulgaris.json
valerian.json / valeriana-officinalis.json
```

Fix: keep the Latin name as canonical, add the common name as a URL alias or redirect in public/_redirects.

10. Sitemap has 461 herb routes vs 465 in the publication index

Four herbs that pass the quality gate are missing from sitemap.xml. Likely a generation order issue — the sitemap script runs before the indexable-herbs file is fully updated. Reorder the postbuild step or regenerate the sitemap after indexable-herbs.json is finalized.

11. robots.txt disallows /herb-index which is just a redirect

The route /herb-index immediately redirects to /herbs via a Navigate component in App.tsx. Disallowing it in robots.txt is harmless but creates noise. More importantly: /blog/page/:page (pagination routes) and /collections/:slug are not explicitly allowed, which may cause Googlebot to under-crawl paginated blog results.

12. Two separate blog content systems

There are **10 Markdown files in src/data/blog/** and **75+ MDX files in content/blog/**. Only the content/blog/ files go through the build:blog pipeline. The src/data/blog/ files appear to be a legacy system — if they are wired into the blog router they could cause duplicates; if not wired in, they are dead content.

■ Architecture & SEO Concerns

13. Greek-letter slug mismatch between pipeline and UI

The **quality-gate-data.mjs** script has its own slugify function that maps Greek letters to spelled-out names (e.g. alpha → "alpha-", beta → "beta-") before normalization. The UI's **src/lib/slug.ts** does plain ASCII lowercasing with no Greek letter handling. Compounds like alpha-asarone or beta-carbolines may end up with different slugs in the data pipeline vs what the router expects, causing 404s on Greek-letter compound pages.

```
Fix: unify the slugify logic. Copy the Greek letter map into src/lib/slug.ts.
```

14. ops/ directory ships to production

The **ops/** directory contains 50+ report JSONs, enrichment manifests, wave planning files, PDF research documents, and contractor notes — all committed to the repo and therefore included in the Cloudflare Pages deploy. None of this belongs in a production website. It adds deploy weight, exposes internal operations data publicly, and clutters the repo. Move ops/ to .gitignore or a separate branch.

15. npm run build bypasses prebuild if called via build:compile

The build script is defined as **"build": "npm run build:compile"**. When npm runs **npm run build**, the lifecycle prebuild hook should fire automatically. However, if anyone runs **npm run build:compile** directly (common during debugging), the entire prebuild pipeline is silently skipped — no quality gate, no indexable-herbs regeneration, no sitemap update. This is a latent footgun. Consider renaming build:compile to an internal name or adding a guard comment.

16. No slug sanity check in the prebuild pipeline

The recurring nan/null/[object Object] artifacts from the data pipeline keep reappearing because there is no automated guard catching them before they reach production. Add a simple validation step in prebuild that asserts: no entity slug is under 3 characters, no slug equals 'nan', 'null', 'undefined', or 'object-object', and no slug contains brackets or spaces. This is a 20-line script that would have caught every corruption issue in this audit.

Priority Fix List

#	Sev	Fix	Effort
1	■	Delete nan.json, patch achillea-millefolium name	15 min

2	■	Delete object-object.json compound	2 min
3	■	Delete degenerate compound slugs (5, n, alpha, beta, meo, p)	5 min
4	■	Route or delete HerbBlender, CompoundIndex, Lesson pages	1 hr
5	■	Fix blog build pipeline — 60 posts not published	1–2 hr
6	■	Audit verify-publishing.mjs for manifest structure mismatch	30 min
7	■	Audit 212 unpublished herbs — many near-ready	2–4 hr
8	■	Move LSD/THC/CBD/psilocybin from herbs to compounds	30 min
9	■	Deduplicate common-name alias herb files	1 hr
10	■	Fix sitemap generation order (461 vs 465 herbs)	30 min
11	■	Unify Greek-letter slugify between pipeline and UI	45 min
12	■	Exclude ops/ from git / Cloudflare deploy	15 min
13	■	Add slug sanity check to prebuild pipeline	1 hr
14	■	Guard against build:compile bypassing prebuild	15 min

What's Working Well

Data architecture: The separation between raw detail files, summary layer, quality gate, and publication index is genuinely solid. The normalization philosophy (UI adapts to imperfect data) is the right call at this scale. Don't change it.

Routing: BrowserRouter is correct, lazy loading is properly implemented, error boundaries are in place. The legacy redirect pattern for /herb/:slug is clean.

Security headers: The _headers file is comprehensive — HSTS, CSP, frame denial, CORP all set correctly. The CSP unsafe-inline note is properly documented.

Duplicate slugify: The old duplicate-slugify bug is resolved. src/utils/slugify.ts correctly re-exports from src/lib/slug.ts. The Greek letter gap in the pipeline is a separate issue.

Deploy workflow: The GitHub Actions deploy workflow is well-structured — enrichment release gate, asset verification, and Cloudflare wrangler deploy are all in the right order.

Quality gate thresholds: The current thresholds (20-char description, 1 source minimum) are appropriately permissive given your data quality goals. They're catching real garbage without over-filtering.
